Scoring Combine-agent answers: RAGAS vs rubric vs key points

Chirayu Gupta

Vrije Universiteit Brussel

October 2026

RAGAS factual correctness disagrees with the rubric

`gmn-control-region-1`, mean of 3 runs per configuration

agent	FC recall	rubric (1–5)
GPT-5.6 luna, 3 configs	0.43–0.55	5.0
GPT-5.6 terra, 3 configs	0.41–0.66	5.0
Qwen 3.8, changed config	0.55	3.7

- 37 answers got rubric 5/5; their FC ranges 0.22 to 0.92
- reading them: the GPT answers are correct

What `FactualCorrectness(mode="recall")` computes
(ragas 0.2.15, `_factual_correctness.py`)

$$\text{recall} = \frac{TP}{TP + FN}$$

- TP: claims split from the **answer** that the gold supports
- FN: claims split from the **gold** missing in the answer
- two different LLM splits, redone on every call
- every gold claim weighs the same, side remarks included
  (e.g. the docs' "∼50%" comment in gmn)

Not "fraction of gold covered": it moves with answer length and claim split

Replacement: per-question key points ("nuggets")

Written once per question, frozen, versioned (`nuggets.yaml`):

`gmn-control-region-1`	
vital	use the `gmN` uncertainty type
vital	CR count N written after the keyword
vital	extrapolation factor α in the process column
vital	rate $= N\alpha$
okay	concrete datacard line; $1/\sqrt{N+1} \approx 45\%$
must_not	recommends `lnN`/`lnU`; example card inconsistent

Judge (Qwen 3.8 on aigw) per item:

- supported / partly / not supported
- must quote the answer; harness checks the quote is really there
- gold prose not shown; items shuffled; 3 judge runs, majority

Scores per answer:

$$V_{\text{strict}} = \frac{\#\,\text{vital fully supported}}{\#\,\text{vital}}$$

pass = all vital supported **and** no must_not present

Method of the TREC 2024/25 RAG track (AutoNuggetizer), plus evidence quotes and must_not items

Results: 2 questions $\times$ 8 agent configurations $\times$ 3 runs

question	agent / config	nugget pass	V_{strict}	FC recall	rubric
gmn	Qwen vanilla	0/3	0.00	0.00	1.0
	Qwen changed	2/3	0.67	0.55	3.7
	luna (3 configs)	3/3, 3/3, 3/3	1.00	0.43–0.55	5.0
	terra (3 configs)	2/3, 3/3, 3/3	1.00	0.41–0.66	5.0
shape-counting	Qwen vanilla	0/3	0.00	0.00	1.0
	Qwen changed	1/3	0.33	0.44	2.7
	luna van./v1/chg.	2/3, 1/3, 3/3	0.67, 0.50, 1.00	0.48, 0.60, 0.84	3.7, 3.7, 5.0
	terra van./v1/chg.	3/3, 3/3, 2/3	1.00, 1.00, 0.83	0.85, 0.80, 0.84	5.0

vanilla: shipped config (bash `ask`); v1: bash `deny` + read-only allow-list; changed: v1 + web fetch/search denied. Killed or empty runs count as 0.

- per answer: nugget pass agrees with "rubric ≥ 4" on 45/48 answers, with "FC ≥ 0.5" on 38/48
- correct GPT answers on gmn: $V_{\text{strict}} = 1.00$ where FC gave 0.41–0.55

Key points used on the next slides

gmn-control-region-1

kp1 v	use the `gmN` uncertainty type
kp2 v	CR count N written after the keyword
kp3 v	extrapolation factor α in the process column
kp4 v	rate $= N\alpha$
kp5 o	concrete datacard line
kp6 o	rel. uncertainty $\approx 1/\sqrt{N+1}$
mn1 x	recommends `lnN`/`lnU` instead
mn2 x	number after `gmN` is α or the yield
mn3 x	example card inconsistent (rate $\neq N\alpha$, or no N)

agent-shape-counting-1

kp1 v	counting card has no `shapes` line, so no mapping for that channel
kp2 v	fix: `combineCards.py -S`, or `shapes * * FAKE` in the counting card
kp3 o	error "Shape map has no entry for channel 'counting' " at workspace building
kp4 o	mentions both fixes
mn1 x	shape and counting cards cannot be combined at all
mn2 x	blames another cause (name clash, ROOT path, . . .)

v vital, o okay, x must_not. Written from the gold answers; the judge sees only these lines, not the gold prose.

Whole answer: GPT-5.6 luna, changed config, gmn run 2 (1/2)

FC 0.22 rubric 5 nugget pass: kp1–kp5 supported, no must_not

Final answer as stored by the harness; blank lines dropped, long lines wrapped. blue[kp]: supports a key point, orange[mn]: must_not statement.

````
Use a **`gmN`** nuisance parameter[kp1]. It models the Poisson statistical uncertainty from the finite
number of events in the control region, rather than approximating it with a symmetric `lnN`
uncertainty.
If the control region contains \(N\) events and the extrapolation factor to the signal region is
\(\alpha\), write:
```text
name        gmN   N      ...
```
The entry for the relevant background process is \(\alpha\):
```text
bin          signal
process      signal  ttbar
process      0       1
rate         10.0    6.0
crStat       gmN     20      -       0.30[kp5]
```
Here:
- `20` is the observed control-region count[kp2].
- `0.30` is the extrapolation factor[kp3].
````

Whole answer: GPT-5.6 luna, changed config, gmn run 2 (2/2)

FC 0.22 rubric 5 nugget pass: kp1–kp5 supported, no must_not

Final answer as stored by the harness; blank lines dropped, long lines wrapped. blue[kp]: supports a key point, orange[mn]: must_not statement.

```
- The nominal signal-region background yield is therefore \(20 \times 0.30 = 6.0\)[kp4].
- `-` means the nuisance does not affect the signal.
Do **not** enter `1/sqrt(N)` as an `lnN` value for this case. The `gmN` constraint retains the
correct Poisson behavior, especially for small control-region counts. Any separate uncertainty on
the extrapolation factor itself should generally be represented by its own nuisance, often an
`lnN`.
See the Combine documentation's [simple counting experiment](https://cms-
analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/#a-simple-
counting-experiment), which specifically recommends this treatment for backgrounds determined from
control-region event counts and an extrapolation factor.
```

Whole answer: GPT-5.6 terra, vanilla config, gmn run 2 (1/2)

FC 0.50 rubric 5 nugget fail: kp1–kp5 supported, but must_not mn3 present

Final answer as stored by the harness; blank lines dropped, long lines wrapped. blue[kp]: supports a key point, orange[mn]: must_not statement.

````
Use a **gamma-distributed (`gmN`)[kp1]** nuisance. It is specifically intended for a yield inferred
from a finite control-region count (or limited-MC count) multiplied by a transfer/extrapolation
factor.
If the control region contains \(N\) observed events and the signal-region extrapolation factor is
\(\alpha\), write the nuisance row as:
```text
bkg_stat  gmN  N  -  alpha[kp5]
```
Here `N` is the integer CR count[kp2], and `alpha` is entered in the column of the background process[kp3]
whose signal-region rate is \(N\alpha\)[kp4]. For example, 25 CR events with \(\alpha=0.12\):
```text
process        sig  bkg
process        0    1
rate           3.0  3.0
bkg_stat       gmN  -    0.12[mn3]
```
The nominal background yield is \(25 \times 0.12 = 3.0\), and Combine uses the Poisson statistical
uncertainty from the 25-event control count. Do not use `lnN` for that counting uncertainty; use
`lnN` separately only if the extrapolation factor itself has an independent relative systematic
````

Whole answer: GPT-5.6 terra, vanilla config, gmn run 2 (2/2)

FC 0.50 rubric 5 nugget fail: kp1–kp5 supported, but must_not mn3 present

Final answer as stored by the harness; blank lines dropped, long lines wrapped. blue[kp]: supports a key point, orange[mn]: must_not statement.

```
uncertainty.
See the official [simple counting-experiment documentation](https://cms-
analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/), which gives
this control-region/extrapolation-factor `gmN` construction.
```

Whole answer: Qwen 3.8, changed config, shape + counting run 1 (1/2)

FC 0.48 rubric 2 nugget fail: 0/2 vital points, must_not mn1 present (answered with no tool calls)

Final answer as stored by the harness; blank lines dropped, long lines wrapped. blue[kp]: supports a key point, orange[mn]: must_not statement.

```
The problem is that you're trying to combine a **shapes** datacard and a **counting** datacard
using `combineCards.py`.
These two datacard formats are fundamentally incompatible[mn1]:
- **Counting datacards** (like `realistic-counting-experiment.txt`) use simple integer counts and
specify uncertainties via `lnN`, `gmN`, etc. They don't reference ROOT files.
- **Shapes datacards** (like `simple-shapes-TH1.txt`) reference ROOT files containing histograms
(`$PROCESS` and `$PROCESS_$SYSTEMATIC`) and use `shape` uncertainties. They require
`text2workspace.py` to build a workspace with histogram bins.
`combineCards.py` can only combine datacards of the **same type**[mn1] (all counting or all shapes).
Mixing them produces a malformed datacard that `combine` cannot parse or execute.
**Solution:**
You need to convert both analyses to the **same format**. Typically, this means:
1. **Convert the counting experiment to a shapes datacard** by creating a ROOT file[mn1] with the
expected signal/background histograms (even if they're flat/single-bin) and rewriting the counting
datacard in shapes format, OR
2. **Convert the shapes experiment to a counting datacard** if the shapes are simple enough (e.g.,
single-bin), though this loses the shape information.
The most common approach is to express both as **shapes datacards** with corresponding ROOT files,
then combine them with `combineCards.py`.
```

Whole answer: Qwen 3.8, changed config, shape + counting run 1 (2/2)

FC 0.48 rubric 2 nugget fail: 0/2 vital points, must_not mn1 present (answered with no tool calls)

Final answer as stored by the harness; blank lines dropped, long lines wrapped. blue[kp]: supports a key point, orange[mn]: must_not statement.

```
Alternatively, if you want to keep them separate, run `combine` on each datacard independently and
combine the results at the post-processing level (e.g., using `CombineHarvester` or manual
combination of limit results), but this is less straightforward and not the standard workflow.
**To verify:** You can inspect the output of `combineCards.py` -- it will likely produce a datacard
with inconsistent `shapes` and `bin`/`process` sections that `combine` rejects.
```

Whole answer: GPT-5.6 luna, changed config, shape + counting run 1 (1/3)

FC 0.89 rubric 5 nugget pass: kp1–kp4 supported, no must_not (opening lines: narration, joined into the answer by the harness)

Final answer as stored by the harness; blank lines dropped, long lines wrapped. blue[kp]: supports a key point, orange[mn]: must_not statement.

```
I'll inspect the combination semantics and reproduce the failure path, focusing on how
`combineCards.py` namespaces channels and shape references across the two cards.I'm checking the
combined-card mechanics and reproducing the failure so the diagnosis distinguishes a datacard
incompatibility from a missing/invalid shape input.The likely issue is in how `combineCards.py`
rewrites the shape-based card, but I'll verify the generated card and the actual Combine error
before narrowing it down.The combined card shows the key distinction: it contains the shape-file
directive for the `shapes` channel, but no `FAKE` directive for the counting channel. I'll
reproduce the failure locally, then compare it with the documented `-S` conversion path.The
generated card is syntactically produced, but it lacks a shape mapping for the counting channel. I
reproduced the failure: Combine builds the shape model for `counting`, then finds no entry because
the original `combineCards.py` invocation did not request conversion of the counting card to a
fake shape card. I'm verifying the corrected combined card against the actual ROOT input now.The
combined datacard contains two different channel types:
- `shapes` is a shape-based channel and has a `shapes` mapping to `simple-shapes-TH1.root`.
- `counting` is a counting-only channel and has no `shapes` mapping[kp1].
`combineCards.py` allows the combination, but the resulting card is not valid for
`text2workspace.py`/`FitDiagnostics`. Combine tries to load shapes for the `counting` channel and
fails with:
```text
```
```

Whole answer: GPT-5.6 luna, changed config, shape + counting run 1 (2/3)

FC 0.89 rubric 5 nugget pass: kp1–kp4 supported, no must_not (opening lines: narration, joined into the answer by the harness)

Final answer as stored by the harness; blank lines dropped, long lines wrapped. blue[kp]: supports a key point, orange[mn]: must_not statement.

````
KeyError: "Shape map has no entry for channel 'counting'"[kp3]
```
Convert the counting card to an equivalent shape card by adding:
```text
shapes * * FAKE[kp2]
```
to `realistic-counting-experiment.txt`, or use `combineCards.py -S`[kp4]:
```bash
combineCards.py -S \
  shapes=attachments/simple-shapes-TH1.txt \
  counting=attachments/realistic-counting-experiment.txt \
  > combined.txt
combine -M FitDiagnostics combined.txt
```
The `-S` option inserts the required fake-shape mapping for counting channels. This is the
documented solution: [Combination of multiple datacards](https://cms-
analysis.github.io/HiggsAnalysis-CombinedLimit/latest/part2/settinguptheanalysis/#combination-of-
multiple-datacards).
The existing `combined-shapes.txt` demonstrates the corrected structure:
````
````

Whole answer: GPT-5.6 luna, changed config, shape + counting run 1 (3/3)

FC 0.89 rubric 5 nugget pass: kp1–kp4 supported, no must_not (opening lines: narration, joined into the answer by the harness)

Final answer as stored by the harness; blank lines dropped, long lines wrapped. blue[kp]: supports a key point, orange[mn]: must_not statement.

````
```text
shapes *          counting  FAKE
shapes *          shapes    simple-shapes-TH1.root $PROCESS $PROCESS_$SYSTEMATIC
```
I reproduced the failure with the original combined card and confirmed that the `-S` version runs
through `FitDiagnostics`.
````

What the nuggets add: a named reason for every failure

Caught, rubric said 5/5

- terra vanilla, gmn run 2: example card `bkg_stat gmN - 0.12` (no *N* after gmN) → must_not "inconsistent example" present

Diagnosed

- killed luna runs (permission reject, path typo): miss both vital points, not "low recall"
- Qwen changed, shape run 1: "shape and counting cards cannot be combined" → must_not

Missed / wrong (judge limits)

- terra changed, gmn run 2: `rate 10.0` with `gmN 25 0.12` ($25 \times 0.12 = 3.0$): arithmetic not caught
- 2 correct shape-counting answers failed: judge padded its quote with text not in the answer, quote check rejected it

rubric-vs-nugget disagreements	3/48
real error found by nuggets	1
judge false negative	2

Caveats and next steps

Caveats

- only 2 questions; key points written after seeing answers
- not yet calibrated against human labels
- Qwen judge grades Qwen answers (self-preference)
- 2 vital points on shape-counting: one judge flip $= \pm 0.5$
- agreement with humans is good for ranking systems, weak per question (TREC: τ 0.73–0.90 vs 0.3–0.66)

Next

- key points for all questions, drafted automatically from the **gold only** (LLM ensemble), then reviewed
- must_not items mined from observed wrong answers
- one-off calibration vs the 13 Oct. hand grades (Scott's π, judge leniency)
- cross-family judge for headline numbers
- report pass@1, pass^3, errors clustered by question

Proposal: nugget pass and V_{strict} as headline, rubric as second opinion, drop FC recall

References (scoring)

- E. Voorhees, *Overview of the TREC 2003 QA track*: vital/okay nuggets
- J. Lin, D. Demner-Fushman, *Will pyramids built of nuggets topple over?*, HLT-NAACL 2006
- R. Pradeep et al., *Initial nugget evaluation results for the TREC 2024 RAG track (AutoNuggetizer)*, arXiv:2411.09607
- R. Pradeep et al., *The Great Nugget Recall*, SIGIR 2025, arXiv:2504.15068
- S. Upadhyay et al., *Overview of the TREC 2025 RAG track*, arXiv:2603.09891
- R. Arora et al., *HealthBench*, arXiv:2505.08775 A. Gunjal et al., *Rubrics as Rewards*, arXiv:2507.17746
- Y. Lee et al., *CheckEval*, EMNLP 2025, arXiv:2403.18771
- M. Wanner et al., *A closer look at claim decomposition*, arXiv:2403.11903
- A. S. Thakur et al., *Judging the judges*, arXiv:2406.12624 A. Panickssery et al., arXiv:2404.13076
- S. Yao et al., τ-*bench* (pass^k), arXiv:2406.12045 E. Miller, *Adding error bars to evals*, arXiv:2411.00640